

机器学习基础 Machine Learning

实验二：BP 算法

重庆大学计算机学院
王翊

2026年5月

随着人工智能的发展，词元调用量呈现出指数级增长。截至3月底，日均值已突破140万亿，和2024年底相比增长超千倍。

一、算力单位换算

算力进制：

$$1 \text{ EFLOPS} = 10^3 \text{ PFLOPS} = 10^6 \text{ TFLOPS}$$

$$\begin{aligned} 870 \text{ EFLOPS} &= 870000 \text{ PFLOPS} \\ &= 870000000 \text{ TFLOPS} \end{aligned}$$

二、单张 RTX 4090 算力 (AI 常用 FP16)

RTX 4090 **FP16** 张量算力：

$$\text{单张4090} \approx 191 \text{ TFLOPS}$$

三、870 EFLOPS 等价多少张 4090

$$\text{显卡数量} = \frac{870 \times 10^6 \text{ TFLOPS}}{191 \text{ TFLOPS/张}} \approx 455.50 \text{ 万张}$$

□ 要素

- 1 数据集：标签-参数对，形成一个矩阵
- 2 模型（训练）：
 - 1) 模型结构： 输入层、隐藏层、输出层，参数
 - 2) 模型参数： $W1, W2, b1, b2$
 - 3) 损失函数：交叉熵损失
 - 4) 最优化算法：梯度下降+反向传播
 - 5) 训练
- 3 应用（推理）

数据集-Iris

X (4 个特征)

y (共有 3 类)

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3	1.4	0.1	setosa
14	4.3	3	1.1	0.1	setosa
15	5.8	4	1.2	0.2	setosa
16	5.7	4.4	1.5	0.4	setosa

□ 模型结构



代表输入的数据

例如 $X = [x_1, x_2, x_3, x_4]^T$

□ 包含三个元素:

- 权重 W
- 偏置 b
- 激活函数(例如 sigmoid)

□ 这一层的计算过程为:

$$\begin{aligned} Z_h &= X \cdot W_h + b_h \\ A_h &= \sigma(Z_h) \end{aligned}$$

□ 包含三个元素:

- 权重 W
- 偏置 b
- 激活函数(例如 sigmoid)

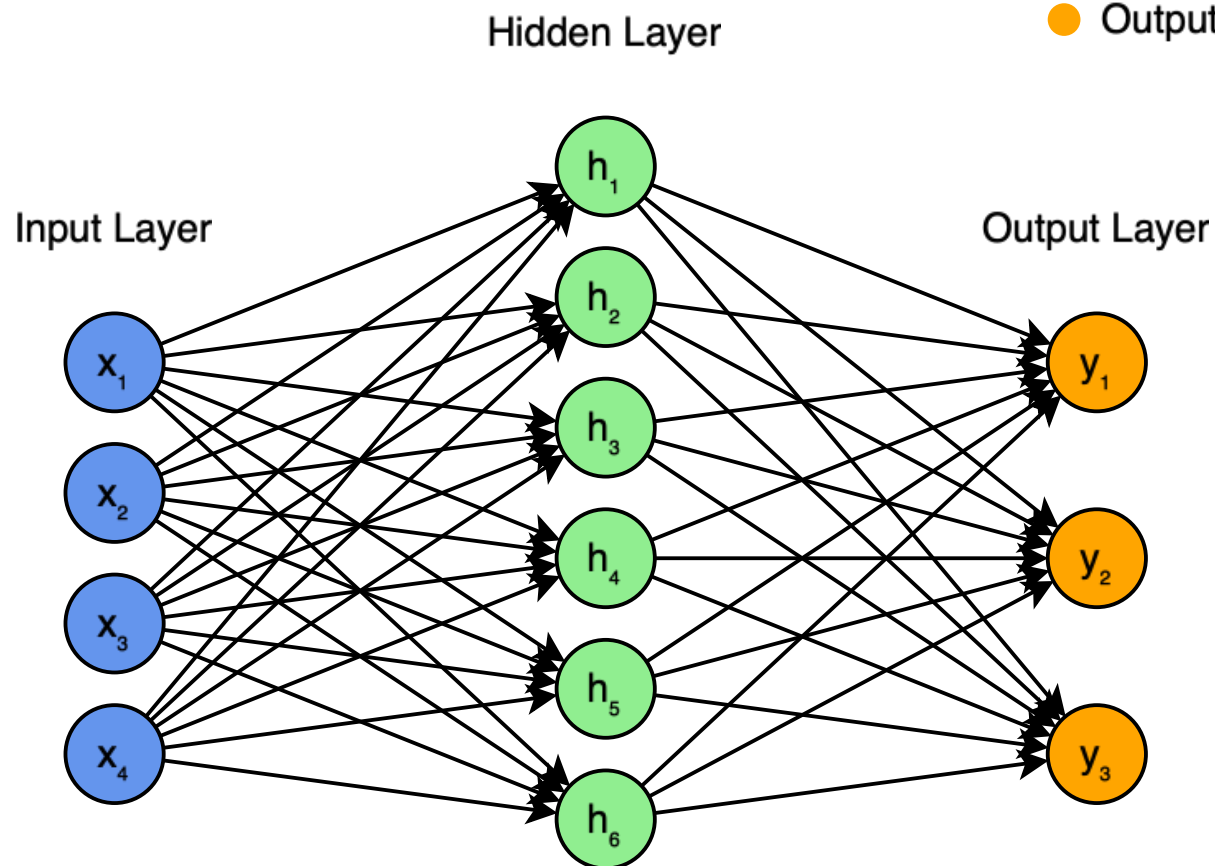
□ 这一层的计算过程为:

$$\begin{aligned} Z_o &= A_h \cdot W_o + b_o \\ A_o &= \sigma(Z_o) \end{aligned}$$

输入层是没有任何计算的，只代表输入的数据；隐藏层可以多次堆叠，即为深度神经网络

□ 模型-例子

- Input Layer
- Hidden Layer
- Output Layer



输入层 $X = [x_1, x_2, x_3, x_4]^T$

隐藏层输出结果:

$$Z_1 = X \cdot W_1 + b_1$$

$$A_1 = \sigma(Z_1)$$

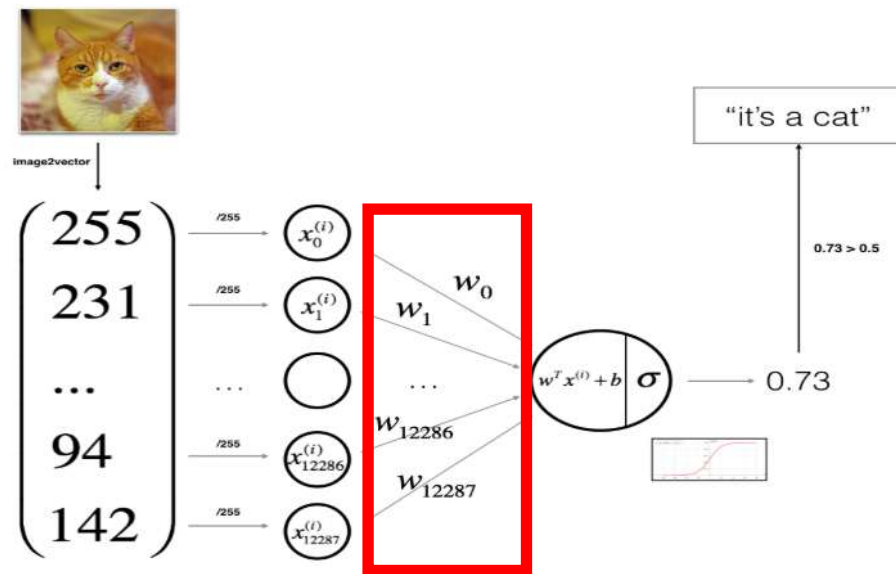
输出层输出结果:

$$Z_2 = A_1 \cdot W_2 + b_2$$

$$A_2 = \sigma(Z_2)$$

$+\Delta W \quad +\Delta b$

$$\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & +\Delta W & w_{22} \\ w_{31} & +\Delta W & w_{32} \\ w_{41} & +\Delta W & w_{42} \\ w_{51} & +\Delta W & w_{52} \\ & +\Delta W & \end{pmatrix} \times \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} + \begin{pmatrix} b_1 & +\Delta b & l_1 \\ b_2 & +\Delta b & l_2 \\ b_3 & +\Delta b & l_3 \\ b_4 & +\Delta b & l_4 \\ b_5 & +\Delta b & l_5 \end{pmatrix} f \begin{pmatrix} l_1 \\ l_2 \\ l_3 \\ l_4 \\ l_5 \end{pmatrix} = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \end{pmatrix}$$



前向传播 → 算损失函数 → 反向传播求梯度 → 梯度下降更新参数 → 循环

2. 反向传播算法

python ^

```
dw = (y_pred - y_true) * x
db = (y_pred - y_true)
```

这就是反向传播

利用链式法则，从后往前求导，算出：

- 权重 w 的梯度 dw
- 偏置 b 的梯度 db

3. 梯度下降法

python ^

```
w = w - lr * dw
b = b - lr * db
```

```
# 1. 模拟简单神经网络: 1个输入x, 1个权重w, 1个偏置b
# 目标: 拟合  $y = 2 * x + 1$ 

# 初始参数
w = 0.0
b = 0.0
# 学习率
lr = 0.1
# 训练数据
x = 2.0
y_true = 5.0 # 真实值: 2*2+1=5

# 迭代训练100次
for i in range(100):
    # ----- 前向传播 -----
    y_pred = w * x + b

    # ----- 1. 损失函数: 均方误差 -----
    loss = 0.5 * (y_pred - y_true) ** 2

    # ----- 2. 反向传播: 手动求导 计算梯度 -----
    # 对w的梯度 dL/dw
    dw = (y_pred - y_true) * x
    # 对b的梯度 dL/db
    db = (y_pred - y_true)

    # ----- 3. 梯度下降: 用梯度更新参数 -----
    w = w - lr * dw
    b = b - lr * db

    if i % 10 == 0:
        print(f"迭代{i:2d} | 损失:{loss:.4f} | w:{w:.4f} | b:{b:.4f}")
```


□ One-hot Encoding

One-hot 编码是一种将分类变量转换为二进制向量的方法

假如我们有三种类别

类别 1

1	0	0
---	---	---

类别 2

0	1	0
---	---	---

类别 3

0	0	1
---	---	---

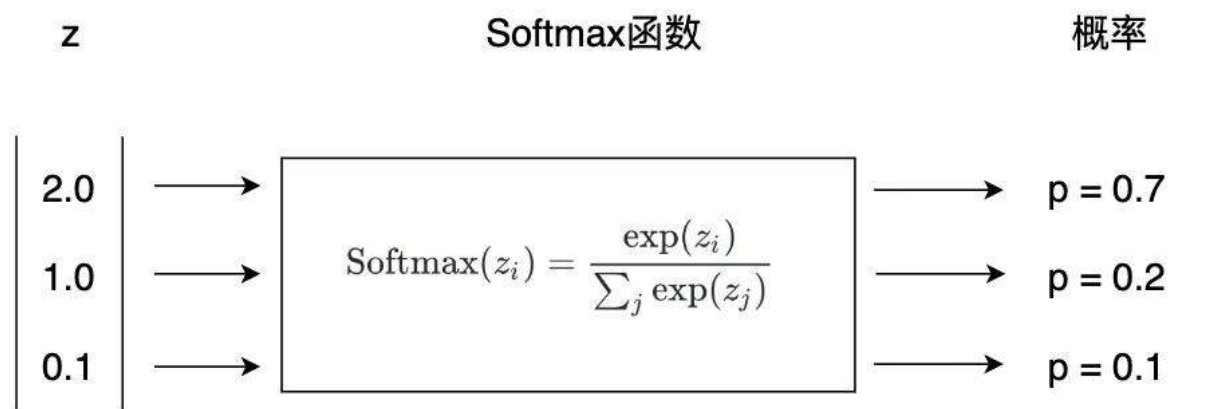
N 位长的 One-hot 编码可以表示 N 个不同类别

每个向量中只有一个位置是 1，其余都是 0

这代表了一个“独热”状态 —— 只有一个神经元被激活（值为 1），其他都为抑制状态（值为 0）

□ 激活函数

Softmax 函数将输入值映射为0-1之间的概率实数，适用于多分类任务



知乎 @PP鲁

例子

输出层输出结果:

$$\mathbf{Z}_2 = \mathbf{A}_1 \cdot \mathbf{W}_2 + \mathbf{b}_2$$

$$\mathbf{A}_2 = \sigma(\mathbf{Z}_1)$$

其中, $\mathbf{Z}_2 = [2.0, 1.0, 0.1]$

则 $\mathbf{A}_2 = [0.7, 0.2, 0.1]$

其中 0.7最大, 所以结果为 **类别 1**

□ 交叉熵损失函数

针对多分类任务：

- 输入样本有 C 个类别
- 神经网络输出是一个概率分布 $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_C]$
- 真实标签是 One-hot 向量 $y = [y_1, y_2, \dots, y_C]$ ，其中 $y_i = 0$ 或 1

那么交叉熵损失函数表达式为：

单个样本： $\mathcal{L}_i = -\sum_{c=1}^C y_{ic} \cdot \log(\hat{y}_{ic})$ ，其中 i 表示第 i 个样本

总体： $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \cdot \log(\hat{y}_{ic})$

```
loss = -np.mean(np.sum(y * np.log(self.a2 + 1e-9), axis=1))
```

□ 最优化算法

梯度下降+反向传播

偏导数计算-链式法则

首先对输出层参数 W_2 , b_2 求偏导

然后对隐藏层参数 W_1 , b_1 求偏导

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_2} = \mathbf{A}_2 - \mathbf{Y}$$

Softmax + Cross-Entropy的联合梯度简化形式
不需要计算损失值也能求梯度

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \frac{1}{m} \mathbf{A}_1^\top (\mathbf{A}_2 - \mathbf{Y})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \frac{1}{m} \sum_{i=1}^m (\mathbf{A}_2^{(i)} - \mathbf{Y}^{(i)})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_1} = (\mathbf{A}_2 - \mathbf{Y}) \mathbf{W}_2^\top$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} = [(\mathbf{A}_2 - \mathbf{Y}) \mathbf{W}_2^\top] \circ \mathbf{A}_1 \circ (1 - \mathbf{A}_1)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \frac{1}{m} \mathbf{X}^\top \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} \right)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \frac{1}{m} \sum_{i=1}^m \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1^{(i)}} \right)$$

□ 训练

超参数:
LR
轮数
Batchsize
.....

```
for i in range(epochs):  
    self.forward(X) # 前向传播 计算输出层输出  $Z_2$   
    self.backward(X, y) # 反向传播 更新模型参数  
    if i % 100 == 0: # 每100轮打印一次损失  
        # 计算交叉熵损失  
        loss = -np.mean(np.sum(y * np.log(self.a2 + 1e-9), axis=1))  
        print(f"Epoch {i}: Loss = {loss:.4f}")
```

□ 推理

```
def predict(self, X):  
    """预测函数  
    Args:  
        X: 输入数据  
    Returns:  
        预测类别  
    """  
  
    probs = self.forward(X)  
    return np.argmax(probs, axis=1)
```

```
def forward(self, X):  
    """前向传播计算  
    Args:  
        X: 输入数据  
    Returns:  
        输出层激活值  
    """  
  
    self.z1 = np.dot(X, self.W1) + self.b1 # 隐藏层线性变换  
    self.a1 = sigmoid(self.z1) # 隐藏层激活值  
    self.z2 = np.dot(self.a1, self.W2) + self.b2 # 输出层线性变换  
    self.a2 = softmax(self.z2) # softmax激活  
    return self.a2
```

超参数:

LR

轮数

Bachsize

.....

□ 调试

- 加断点
- 观察
- 控制台